

Algorithm: Linear Regression (custom Implementation)

Dataset: [Medical Insurance Dataset] - [Dataset](#)

1. Business Understanding

Objective

The objective of this work is to implement Linear Regression from custom by using Gradient Descent; thereafter main comparisons will be drawn against the built-in methods from Scikit-Learn, and the effect of feature scaling on the performance of the model will be studied. The broader aim will be therefore to get into a grounded grasp of Gradient Descent and hyperparameter tuning (like learning rate and epochs) as well as model evaluation using manual performance metrics such as Mean Squared Error (MSE). The challenges would be those in getting an optimal learning rate and epoch selection to achieve fast convergence while ensuring that the algorithm remains stable all through training.

2. Data Understanding

This dataset includes the data about some personal specifics (age, sex, BMI, size of the family, smoking habit), geolocation and how they affected the cost of medical insurance. This information can serve as a basis for understanding the association of these features with insurance cost and creating predictive models. As we have a continuous target variable, this dataset can be used for regression. Feature scaling is important since the features are of different scales.

3. Data Preparation

Feature scaling (Standardization) was applied using 'StandardScaler' from Scikit-learn before training the custom model.

4. Modeling -Baseline Implementation (custom)

Implementation Details:

- Used Gradient Descent for optimization.
- Initial Learning Rate: 0.01
- Initial Epochs: 5000 (configurable)
- Weights initialized to 0.
- Bias initialized to 0.
- MSE is calculated and stored for each epoch.

5. Evaluation

- **Linear Regression (custom) MSE: 0.3573**
- **Linear Regression (Library) MSE: 0.3573**

Observations:

- The MSE values for the custom implementation and Scikit-learn's Linear Regression are very close together, which means that this custom implementation should be functioning correctly.
- The weights learned by the two implementations differ. This is expected, especially since the implementation from custom uses gradient descent.

Summary

- Linear Regression from custom with Gradient Descent has been implemented.
- Feature scaling (Standardization) has been performed correctly and has led to a significant gain in performance.
- MSE logging allows us to analyze the training process.
- Compared performance with scikit-learn's Linear Regression.
- Next up: Deploy the scaler and model trained as a web application. Input should be scaled before we make predictions

Coding implementation Workflow:

Class LinearRegressioncustom:

Initialize with:

- learning_rate (learning rate for gradient descent)
- epochs (number of iterations for training)
- weights (model parameters, initialized to None)
- bias (model bias, initialized to None)
- mse_history (list to store mean squared error for each epoch)

Method fit(X, y):

- Get the number of samples (n_samples) and features (n_features) from X
- Initialize weights as a vector of zeros with length n_features
- Initialize bias as 0
- **For each epoch (from 1 to epochs):**
 - Calculate predicted values: $y_{\text{predicted}} = X \cdot \text{weights} + \text{bias}$
 - Compute gradient with respect to weights: $dw = (1 / n_{\text{samples}}) * X.T. (y_{\text{predicted}} - y)$
 - Compute gradient with respect to bias: $db = (1 / n_{\text{samples}}) * \text{sum}(y_{\text{predicted}} - y)$
 - Update weights using the gradient: $\text{weights} -= \text{learning_rate} * dw$
 - Update bias using the gradient: $\text{bias} -= \text{learning_rate} * db$

- Compute Mean Squared Error (MSE) for this epoch: $mse = \text{mean_squared_error}(y, y_predicted)$
- Append the computed MSE to `mse_history`

Method predict(X):

- Calculate predicted values: $y_predicted = X \cdot \text{weights} + \text{bias}$
- Return the predicted values

Flowchart for implementation

1. Start

- Initialize learning rate, epochs, weights, bias, and an empty list for storing MSE history.

2. Training:

○ Epoch Loop:

- For each epoch:
 - Calculate predicted values ($y_predicted$).
 - Compute the gradient of weights (dw) and bias (db).
 - Update weights and bias.
 - Calculate MSE for the epoch and store it in `mse_history`.

3. Prediction:

- Once training is complete, use the learned weights and bias to make predictions for new data (X).
- Return predictions.